OPENUSD GEOSPATIAL · CO-SUBMISSION PREVIEW

usdGeospatial

running evidence for the Esri geospatial CRS proposal

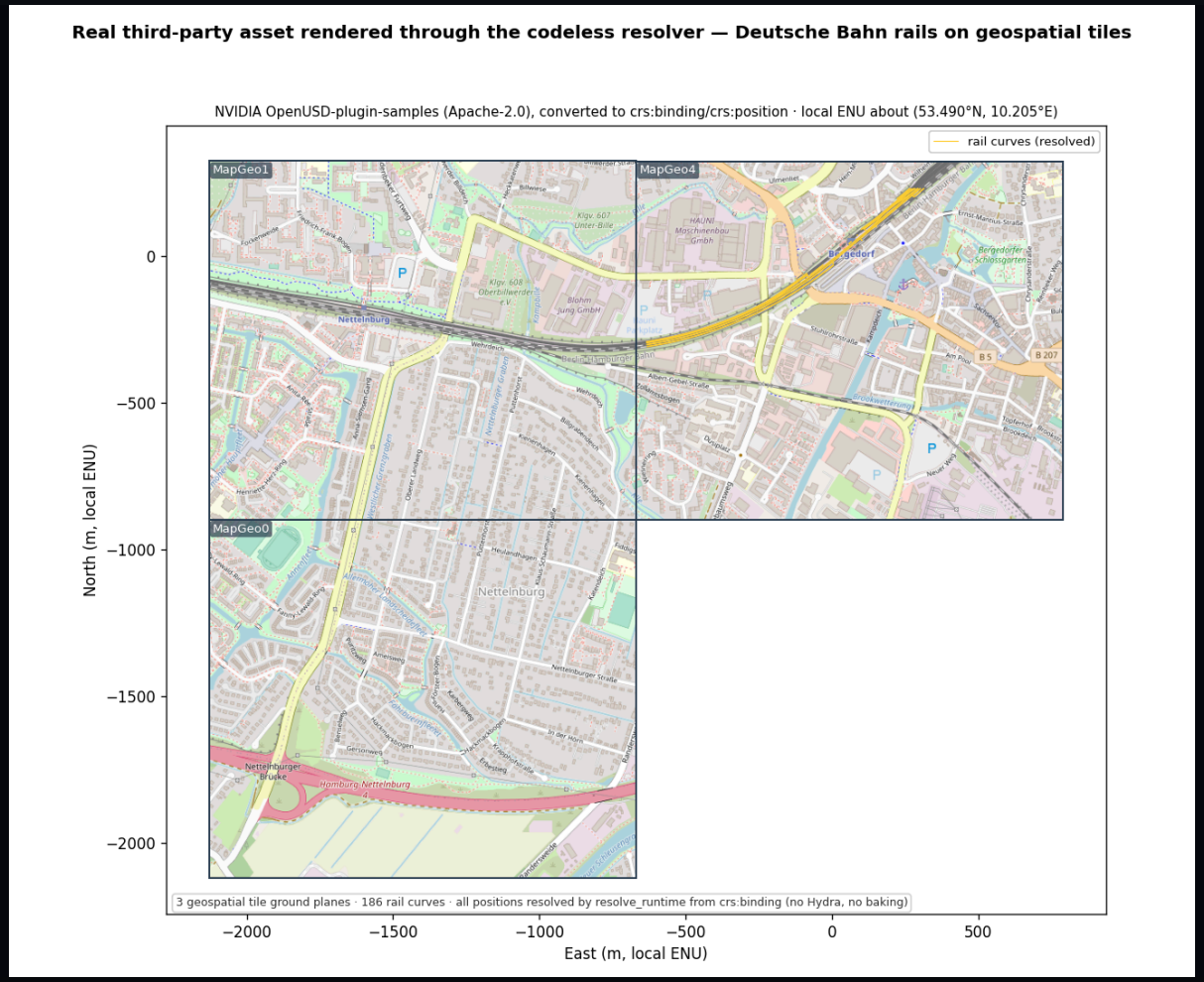
schema declares · runtime resolves · one real-world coordinate frame

The proofs

Four things an independent reviewer can check.

PROOF · NO OVERFIT

Real third-party asset on real tiles



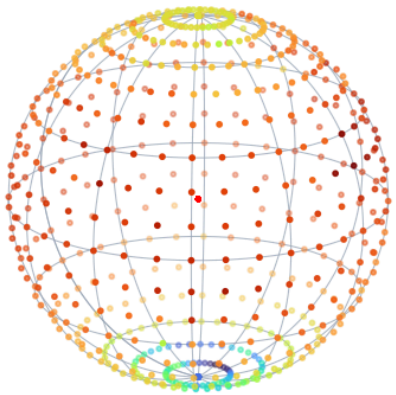
Matplotlib plot (not a render): an external Deutsche Bahn railway (~1,470 curves) authored in the original Omniverse schema, converted to crs:binding/crs:position, resolved onto three real geospatial tiles near Hamburg.

PROOF · CO-REGISTRATION

Three CRSs, one ECEF point

3D coherence — schema-resolved geometry co-registers with closed-form WGS84 ground truth (independent of PROJ)

Earth-2 t2m cloud (resolve_runtime) on a closed-form WGS84 graticule — field sits on the right latitudes



red dots @ origin = same cloud resolved IGNORING crs:binding (negative control)

Cross-CRS co-registration at one benchmark (NOAA NCAT) three CRSs → one ECEF point → matches closed-form GT

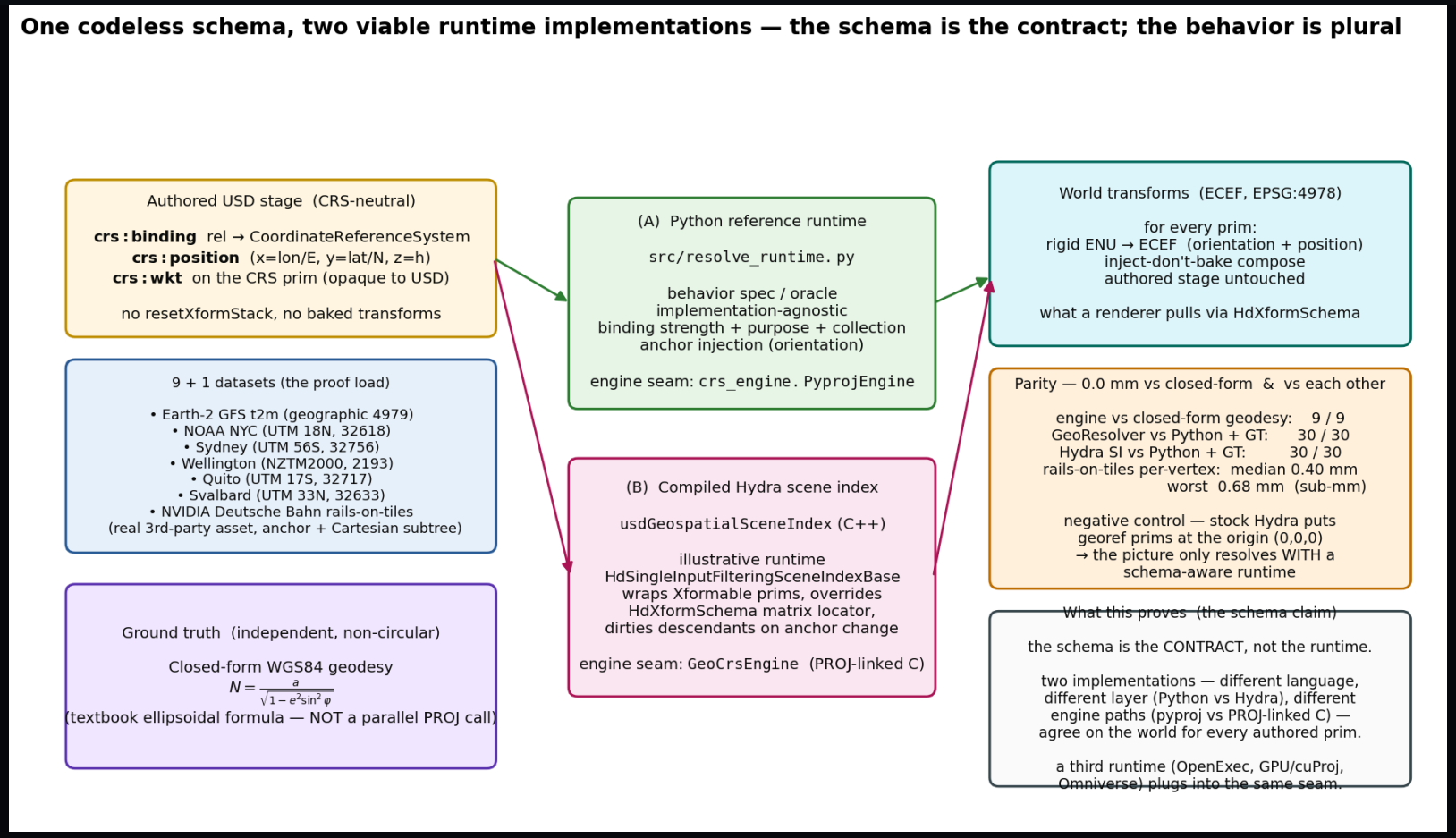
Closed-form WGS84 GT (first principles, no PROJ)	(1334930.899, -4651062.376, 4141331.101)	
SiteGeographic (EPSG:4979) lon/lat from NOAA NCAT		Δ to GT = 0.000 mm
SiteUTM18N (EPSG:32618) E/N from NOAA NCAT		Δ to GT = 0.537 mm
SiteNYSPC (EPSG:32118) State-Plane E/N from NOAA NCAT		Δ to GT = 0.361 mm

three independent CRSs co-register to ≤ 0.537 mm — authored from independent NOAA coords, not by inverting one transform; negative control (ignore binding) lands 6.4×10⁶ m off the globe.

The same monument authored three ways (geographic / UTM 18N / NY State Plane) from independent NOAA NCAT coordinates co-registers to one ECEF point to worst ~0.54 mm — vs closed-form WGS84 geodesy, not a parallel PROJ call. Red blob = negative control (bindings ignored).

PROOF · CONTRACT NOT IMPLEMENTATION

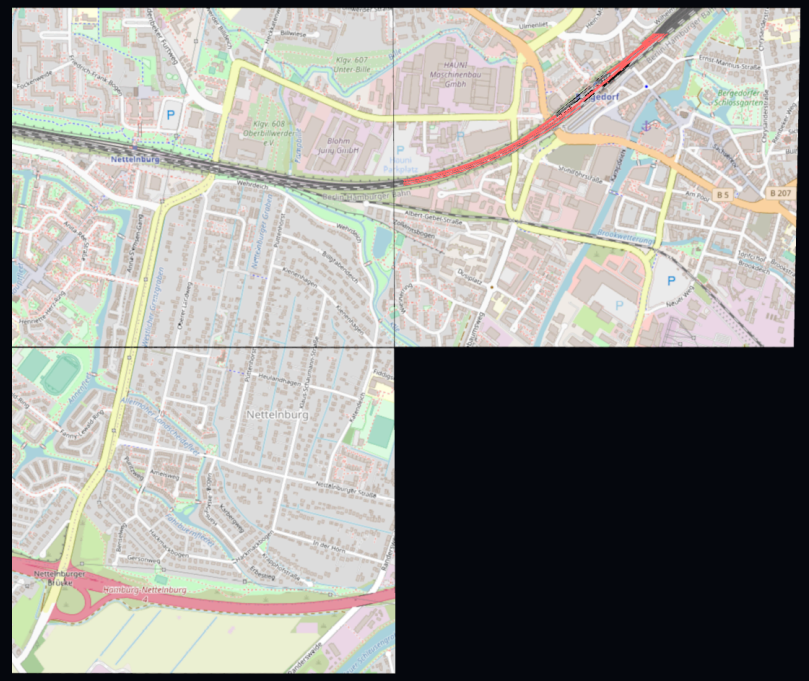
Two independent runtimes, 0.0 mm



Robustness / interoperability pass (correctness is Proof 2 vs NCAT + closed-form geodesy): the Python reference runtime and the compiled C++ Hydra scene index resolve the same authored stage to the same world, agreeing to 0.0 mm — the data contract is unambiguous enough to build twice the same way. A third runtime plugs into the same seam.

PROOF · REAL RENDER, AUTO-INSERT

It just works in usdview (Storm)



Real Hydra Storm render (not a plot): with the plugin on the path, the railway auto-resolves at its ECEF position — no app code. Negative control (plugin removed): railway absent. The auto-insert path matches the reference 30/30 at 0.0 mm (testHydraAutoParity).

The design call

OpenUSD implementation choices that keep the authored scene neutral.

OPENUSD IMPLEMENTATION DETAIL

Codeless schema

- Pure-data schema: a CRS prim + a binding API, `skipCodeGeneration=true` (no compiled types).
- All resolution behavior lives in the example runtimes, not the schema.
- Same shape as the landed Gaussian / particleField contribution: schema + sample runtime(s) + converter + docs.
- Two runtimes here: Python `resolve_runtime.py` and a C++ Hydra scene index that auto-inserts into usdview.

REPLACE · WRAP · OR COEXIST?

Resolve, don't bake

- Binding a CRS must not mutate authored transform data.
- Baking `resetXformStack` into a layer violates that constraint: every downstream consumer inherits it.
- OpenUSD lets this implementation resolve the relationship at runtime while the authored scene stays coordinate-neutral.
- Proven, not argued: the baked Esri scene and our neutral scene land the same corner to 0.0 mm.

COEXISTING WITH THE TRANSFORM HIERARCHY

Anchor injection: inject, don't bake

- The authored scene stays coordinate-neutral; a child remains an ordinary local Cartesian placement relative to a georeferenced anchor.
- The runtime injects the anchor's rigid local-frame→ECEF basis (full ENU orientation) at read time.
- The subtree then combines under it as ordinary `UsdGeomXformable` behavior — nothing baked into the layer.
- `resetXformStack` lives only on a transient, computed Hydra data source, never the authored scene.

COMBINE IN THE CRS-IMPLIED FRAME

Projected vs geographic anchors

- A GEOGRAPHIC/ECEF anchor's child offsets are local metres → combine through the anchor's true-ENU basis (orientation matters; local +Z = ellipsoidal normal).
- A PROJECTED (UTM/State-Plane) anchor's child offsets live in the grid plane → combine IN-PLANE (grid add + reproject), NOT through ENU.
- UTM grid axes differ from true ENU by grid-convergence + point-scale — lifting grid offsets through ENU bends them ~4.86 m over a ~420 m lever.
- Same neutral authored scene; the runtime picks the frame from the bound CRS type. Proven 0.0 mm both ways against closed-form geodesy.

Non-geometric data

A common geospatial case: data with no shape. Visualize it with USD composition arcs — without touching or duplicating the data.

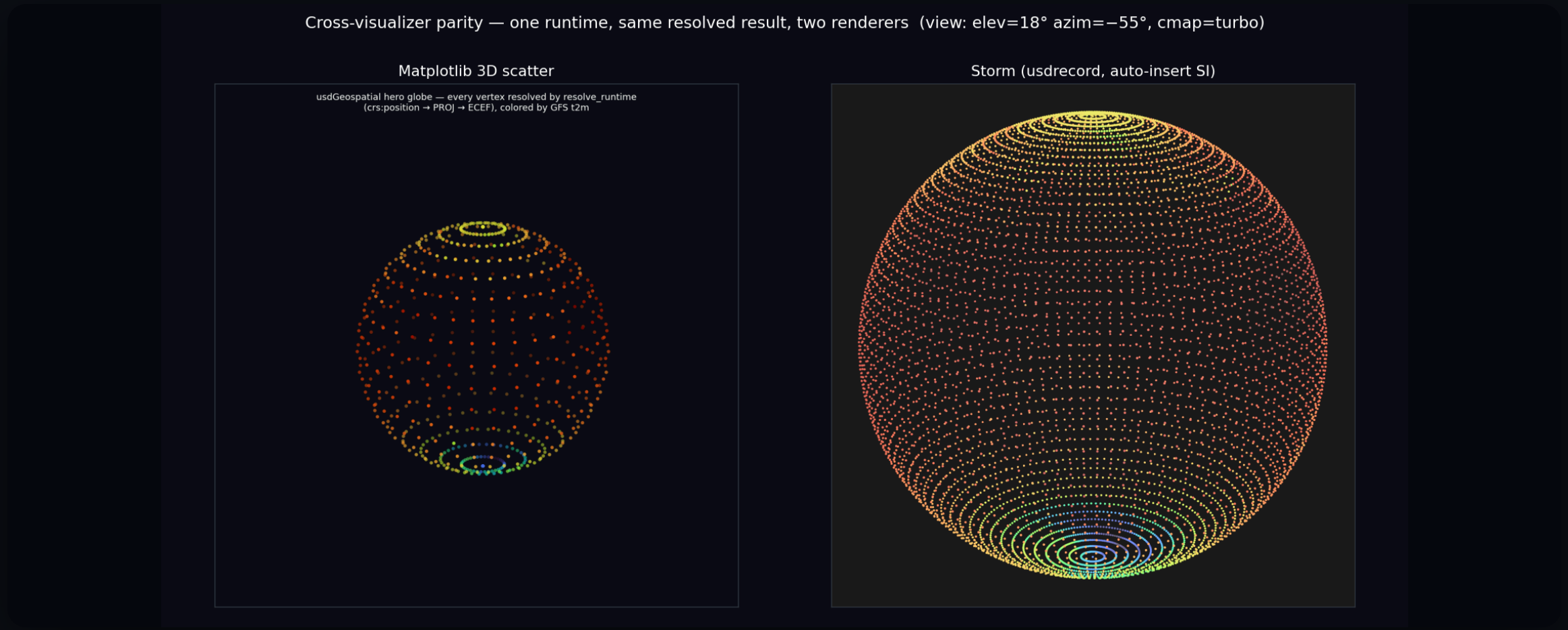
A FIRST-CLASS USE CASE

Non-geometric georeferenced data

- Much geospatial data has NO intrinsic shape: scalar fields (temperature, elevation), sensor/telemetry networks, survey benchmarks. In USD it is a prim carrying `crs:position` + a value, no geometry.
- Visualization is a CHOICE, not part of the data — so add it via USD COMPOSITION ARCS: keep the dataset pristine in a base layer; a separate overlay layer subLayers it and adds marker glyphs as geometry-only `over`s.
- ZERO `crs:*` re-authored; the base stays byte-identical; pull the overlay and you are back to pure data.
- The glyphs are Cartesian children at each prim's local origin — the auto-inserted scene index places them via the SAME anchor-injection path from §7. Marker size is a stated DISPLAY parameter (like a scatter's point size), and the overlay can be sparse.

TWO VISUALIZERS, ONE FIELD

Same non-geometric data, matplotlib vs Storm



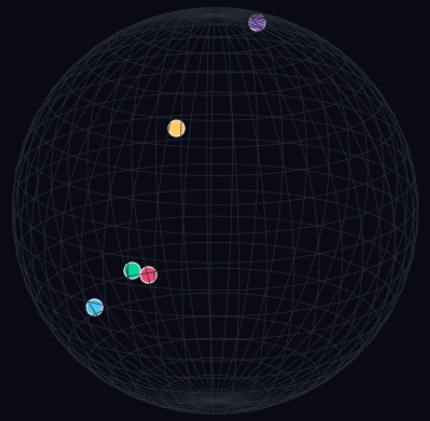
The earth2 GFS t2m field (7,320 crs:position samples, no geometry) visualized two ways: a Matplotlib scatter (left) and a Hydra Storm render of a composition-overlay of marker glyphs (right), placed by the auto-inserted scene index. Same resolved positions; glyph style is a disclosed display choice.

PROOF · NOT OVERFIT

One runtime, five CRS families, both hemispheres

Anti-overfit: one runtime resolves five CRS families across both hemispheres (equator → ~78°N)

Where each authored point resolves (ECEF)



Same code path, five CRS families, zero per-CRS special-casing

One resolution path runs for every locale; the only thing that changes is the authored EPSG. Each point round-trips lon/lat → projected CRS → ECEF through PROJ (the registered default engine).

- NYC — UTM 18N
  `ECEF = (+1.335, -4.651, +4.141) ×10⁶ m`
- Sydney — UTM 56S
  `ECEF = (-4.647, +2.553, -3.533) ×10⁶ m`
- Wellington — NZTM2000
  `ECEF = (-4.780, +0.437, -4.186) ×10⁶ m`
- Quito — UTM 17S (~eq)
  `ECEF = (+1.276, -6.252, -0.020) ×10⁶ m`
- Svalbard — UTM 33N (~78N)
  `ECEF = (+1.258, +0.352, +6.222) ×10⁶ m`

Verified against closed-form WGS84 geodesy in generalization_suite.py (sub-mm, all 5).

Matplotlib plot (not a render): where each locale's authored point resolves in ECEF, through ONE code path — NYC (UTM 18N), Sydney (UTM 56S), Wellington (NZTM2000), Quito (UTM 17S, ~equator), Svalbard (UTM 33N, ~78N). The only thing that changes between locales is the authored EPSG; the five points land 2,200+ km apart on the globe. Verified sub-mm vs closed-form WGS84 geodesy in generalization_suite.py.

Detecting invalid inputs

Neutral authoring keeps CRS intent inspectable, so malformed geospatial inputs can be detected and flagged.

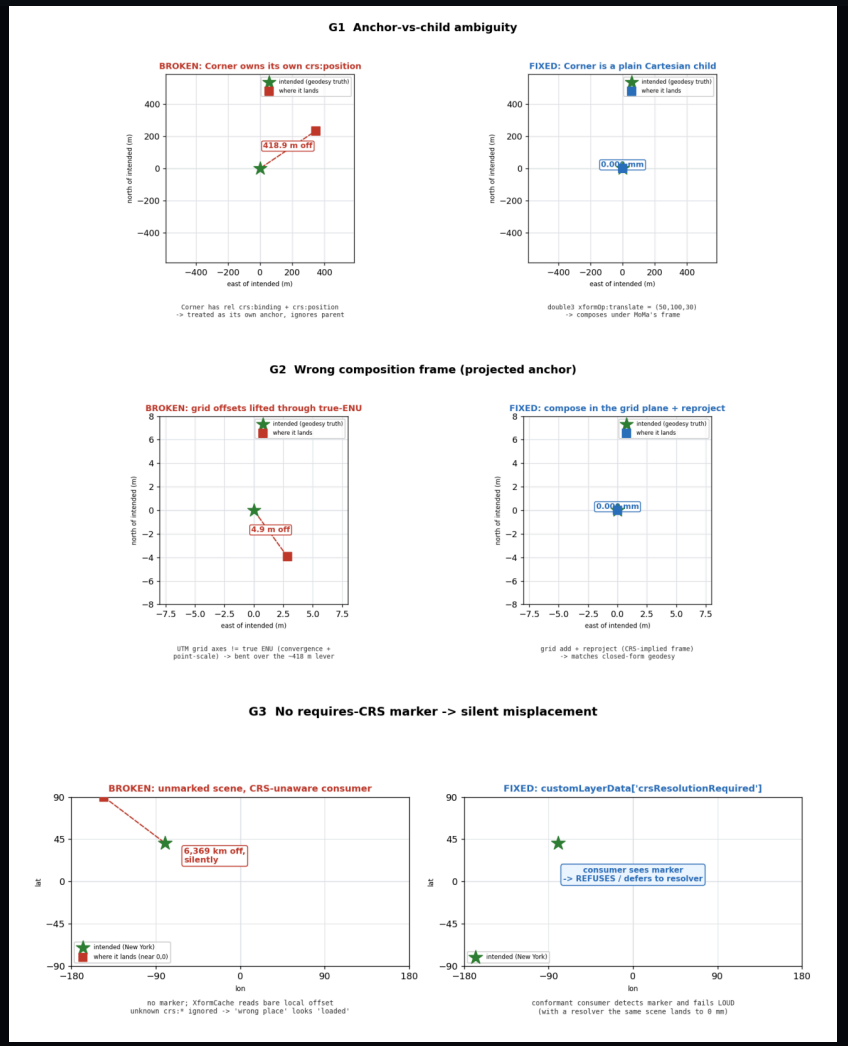
DETECTABLE AND RECOVERABLE

Malformed geospatial inputs can be flagged

- A neutral authored scene PRESERVES the CRS intent a validator needs; a baked scene has already collapsed that intent into a matrix.
- The system can detect malformed geospatial inputs and flag them before they become silent placement errors.
- G1 anchor-vs-child 418.9 m → 0 mm; G2 wrong frame 4.86 m → 0 mm; G3 no marker 6,369 km silent → detect-and-refuse.
- `verify.py` is the validator today; a codeless `usdchecker` plugin is the near-term deliverable.

BROKEN → FIXED, MEASURED

What breaks in coexist, and the fix



G1 anchor-vs-child 418.9 m → 0 mm · G2 wrong frame 4.86 m → 0 mm · G3 no marker 6,369 km silent → detect-and-refuse

ANTICIPATED DEBATE

The marker: a tradeoff worth having

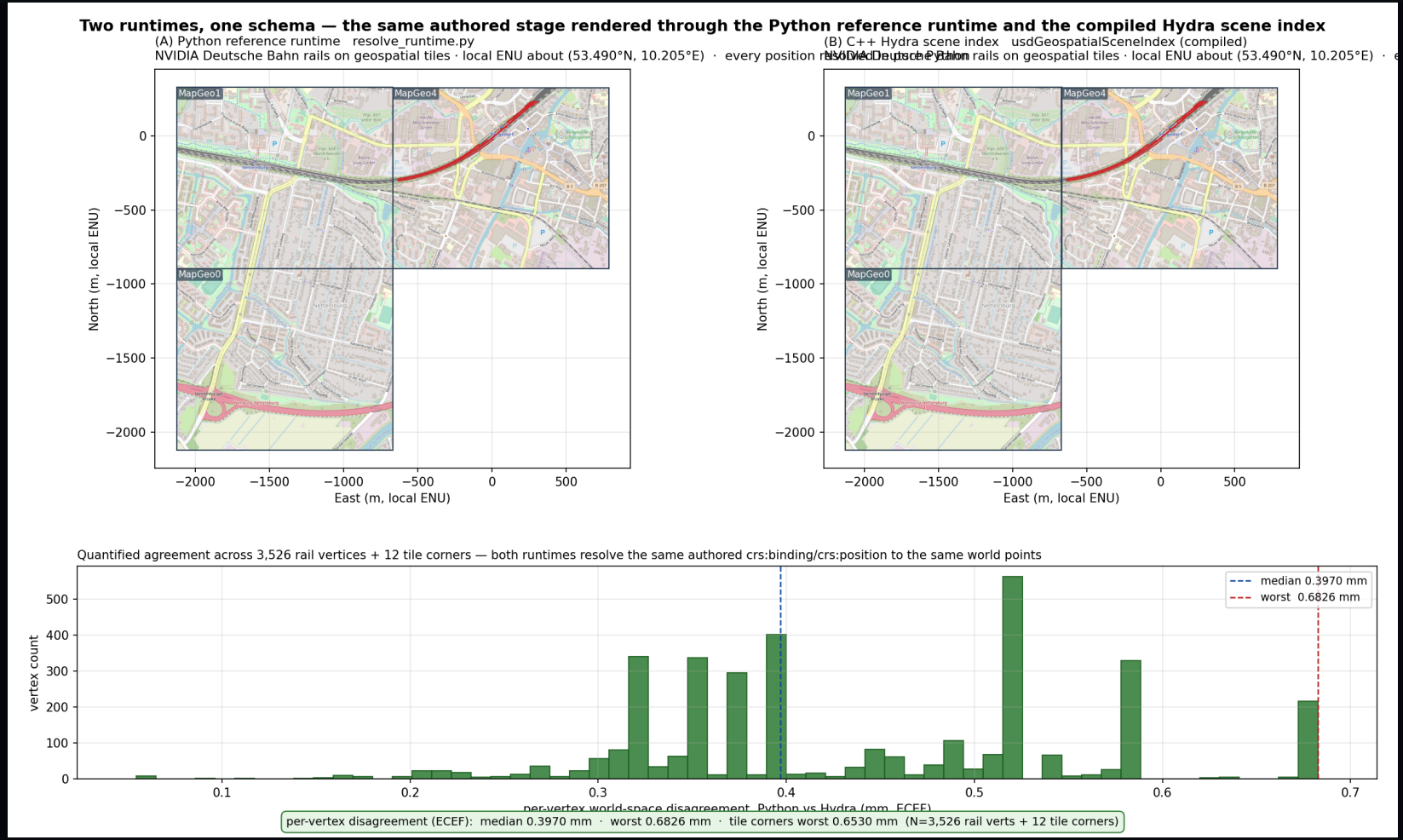
- A 'requires-CRS' marker is the honest cost of coordinate-neutral coexist: it is what lets an unaware consumer fail LOUD instead of silently misplacing.
- Expect pushback — it adds a discovery obligation, and a non-conformant consumer still ignores it (it is a contract, not an enforcement).
- If embraced, the next debate is WHERE it lives: layer metadata (customLayerData) vs a prim-level applied schema.
- Our lean: a stage/layer-level signal for cheap detect-and-refuse, optionally refined per-prim; but this is squarely a WG call.

One schema, two runtimes

The schema is the contract; the behavior is plural.

PROOF · SUB-MM PARITY

Same stage, two runtimes, quantified



Matplotlib plot: Python vs Hydra transforms on the same authored railway stage — 3,526 rail vertices + tile corners agree to median 0.40 mm, worst 0.68 mm. The testUsdGeospatialRuntimeParity ctest fails the build if disagreement exceeds 1 mm.

REPRODUCIBLE · CROSS-PLATFORM

The second runtime builds like any USD example

- `build_usd.py --usdGeospatial` fetches and builds PROJ, then the C++ Hydra plugin, behind one opt-in flag — the `hdParticleField` pattern, no bespoke setup.
- Parity is a `ctest` (`testUsdGeospatialParity`): CRS engine + stage resolver + Hydra scene index + stage-free auto-insert, all 0.0 mm.
- Validated on Linux **and** Windows — the contract builds twice, the same way, on two platforms.

NO EXACT PRECEDENT — BY DESIGN

Where the Python reference runtime sits

- It is a runnable reference: given this authored stage, it demonstrates where each prim should end up, pinned to closed-form geodesy.
- NOT the specification: the normative behavior belongs in prose proposed into the Esri proposal; Python demonstrates it, not replaces it.
- NOT proposed for USD core, NOT a runtime dependency, NOT Python-in-the-render-loop, NOT the prescribed consumer.
- Precedent: AOUSD's core-spec-supplemental — Python sample impls + a compliance framework, spec normative and impl illustrative — is the analogy, not a claim of standard authority.

Open questions

What we'd most like the working group's read on.

FOR THE WORKING GROUP

Open design questions

- 1. Is 'codeless schema + a runnable reference runtime as the behavior contract' the right shape — and where should that reference ultimately live?
- 2. Is **coexist** the right relationship to `UsdGeomXformable` (neutral scene + runtime reconciliation) — versus hooking CRS resolution into `Xformable` directly? Given the head-to-head parity + the guard-rail set, is the residual validation surface acceptable to standardize?

What we need from you

Concrete asks so the next iteration is grounded in your workflows, not our guesses.

ASKS

What we need from you

- 1. The Redlands BIM prototype scene + its validation script, so we can add a second real, contributor-authored case beside the multi-CRS POC.
- 2. Confirmation / correction of the driving workflows: which of AECO site placement, multi-source GIS twins, multi-zone infrastructure, and geodetic sim must 'coexist' survive first?
- 3. A read on the guard-rail set as a conformance surface (anchor-vs-child, compose-in-CRS-frame, a 'requires-CRS' stage marker + detect-and-refuse) — which we intend to enforce via a committed codeless `usdchecker` validator plugin.
- 4. Where the reference runtime should live, and whether to lift the 'resetXformStack' *semantic* out of the authored-layer text into a documented behavior contract multiple runtimes honor.